

Week 13 | Configuration Management

Community Health Survey System (CHSS)

Objective: identify deployment-host prerequisites and express the repeatable target state as an idempotent Ansible playbook.

Evidence date: 8 October 2026 (Asia/Calcutta).

1. Objective and scope

The playbook prepares a Debian/Ubuntu Docker deployment host for the Week 12 CHSS container. It manages packages and Docker Engine state. It does not deploy the app, edit Jenkins, alter the Docker scripts, open firewall ports, or install database/application services.

2. Target node and environment

Development host: Windows. WSL2 enumeration showed only the stopped internal docker-desktop distribution; no Ubuntu/Linux app node was available. Docker CLI could not connect to the Docker Desktop Engine. The inventory is intentionally empty. No target was invented.

Ansible was not installed in the Windows control environment. The first playbook invocation failed before connecting to a target. See `ansible/execution-log.txt` for the captured command output and blocker.

Prerequisite specification

3. Packages

Host packages: docker.io, ca-certificates, and python3. Python

3 must already be available for Ansible module startup. Java 17 JRE

is supplied inside the Docker runtime image (Temurin 17); no host

Java install is needed. Maven 3.9.9 is used in the Docker build stage.

The repository has Maven Wrapper scripts; Maven is not a host runtime dependency. Selenium/browser packages are not target requirements.

4. Users and privileges

No additional Linux application account is required. The container

runs as its own system user chss. Ansible connects as the real

inventory SSH account and uses sudo only for package/service tasks.

No broad Docker group membership is created.

5. Directories and files

The existing Docker deployment needs no host app, config, log, or data

directory. /app and the chss-owned WAR live in the image. The playbook

creates no host files and stores no database or registry credentials.

6. Ports

8081: application port inside the container. 18082: Week 12 host port

mapped to container 8081. This playbook opens no firewall ports.

Jenkins 8082 and Tomcat 8083 are separate Windows services and are not managed on the Linux deployment host.

Services and Ansible design

7. Services and data

Docker Engine is the only host service configured: enabled and started. Jenkins launches the CHSS container; there is no host systemd CHSS service. Week 12 used in-memory H2, so MySQL is not installed or started on this target. Database settings remain external runtime configuration and are never placed in source control by this work.

8. Ansible architecture

Controller: supported Ansible environment. Inventory group: `chss_servers`. Managed node: actual Debian/Ubuntu Docker host, to be added by its operator. `site.yml` gathers facts, checks OS family, installs packages via `apt`, manages Docker via `service`, and checks the engine version with a read-only command. Optional URI check expects HTTP 200 from `/login` if an app is already deployed.

9. Idempotency

`APT state=present` and `service started/enabled` converge to a fixed state. The version command is marked `unchanged`. No shell provisioning, file templates, user churn, or firewall rules are used. An `unchanged` second run should report `changed=0`. No second run was possible in this host.

10. Verification plan

Inventory listing and Ansible ping precede the play. The play verifies OS, module execution, and Docker Engine response. Optional HTTP check verifies the deployed endpoint. Runtime Java 17 evidence comes from the existing Dockerfile/Week 11-12 deployment documentation.

First execution evidence

11. Actual first execution

WSL query: only docker-desktop, state Stopped, WSL version 2.

Docker info: failed to connect to the Docker Desktop Linux Engine;
the named pipe was unavailable.

Command: `ansible-playbook -i inventory.ini site.yml`

Result: PowerShell CommandNotFoundException because ansible-playbook
is not installed. The command failed before playbook parsing or host
connection. This is recorded as BLOCKED, not a successful run.

12. Verification results

No target configuration, package/service checks, app endpoint check,
or idempotency rerun could be performed. YAML and inventory syntax are
validated separately in this deliverable review. The playbook is ready
for a supported controller after a real target is added to inventory.

13. Inventory and execution

Edit `ansible/inventory.ini` with the real Linux host and authorized SSH
user, then run `ansible-inventory --list`, `ansible ping`, and
`ansible-playbook -i inventory.ini site.yml`. To verify an existing
deployment, pass `-e chss_verify_deployed_app=true`.

14. Conclusion

Week 13 configuration is isolated in `chss/ansible`. The specification
matches the actual container deployment and avoids unnecessary host
packages/services. The first run exposed an environment blocker; it
does not claim configuration or verification that did not occur.